ESP32-S3 · ESP-SR OFFLINE SPEECH · AVIONICS PROTOTYPE

CJ2 Voice Emergency Checklist

Technical Data Packet

Cessna Citation CJ2 (CE 525A) — bench / training prototype

Voice-driven, fully offline emergency-checklist reader

Hardware reference, firmware behavior, processor selection, checklist library, and wiring diagram.

DEMO / TRAINING USE ONLY — NOT FOR ACTUAL FLIGHT OPERATIONS

Not airworthy, not certified. Must never be installed in or relied upon aboard an aircraft.
Checklist text reproduced for demonstration/training only.

Repository: github.com/flas-tech/cj2-voice-emergency-checklist

Generated June 10, 2026 · Perplexity Computer

Contents

> DEMO / TRAINING USE ONLY — NOT FOR ACTUAL FLIGHT OPERATIONS. This is a bench/educational prototype. It is not airworthy, not certified, and must never be installed in or relied upon aboard an aircraft.

1 System Overview

An offline, on-device voice-driven emergency checklist reader for the Cessna Citation CJ2 (CE 525A). The pilot states an emergency by name; the device reads each checklist item aloud from pre-recorded audio and waits for the spoken completion word before advancing. It runs entirely on an ESP32-S3 using Espressif's ESP-SR speech framework (WakeNet wake word + MultiNet offline English command recognition) — no cloud, no Wi-Fi, no phone. All GPIO numbers in this packet match `main/board_pins.h`.

Item	Value
MCU	ESP32-S3 (dual-core LX7 @ 240 MHz) — PSRAM required by ESP-SR
Recommended module	ESP32-S3-WROOM-1 N16R8 (16 MB flash, 8 MB octal PSRAM)
Speech stack	Espressif ESP-SR: AFE (NS/VAD) → WakeNet “Hi ESP” → MultiNet English
Mic input	I2S MEMS microphone on I2S_NUM_0
Audio output	I2S Class-D amplifier on I2S_NUM_1 → 4–8 Ω speaker
Config storage	microSD (SDMMC 1-bit), FAT32, one folder per aircraft
Annunciation	Applied Avionics split-legend switch (dark-cockpit, FAA AC 25-11)
Logic level	3.3 V (ESP32-S3 is not 5 V tolerant on GPIO)

Two build paths are supported:

- Integrated: ESP32-S3-Korvo-2 dev board (on-board dual mic, ES8311 codec, NS4150 amp, microSD slot). Least wiring; best mic performance.
- DIY (this document): ESP32-S3 DevKitC-1 N16R8 + INMP441 mic + MAX98357A amp + microSD breakout + the annunciator switch.

2 Master Pin Map

All GPIO assignments from `main/board_pins.h`.

Function	Macro	GPIO	Dir	Notes
Push-to-talk	`PTT_GPIO`	`0`	in (PU)	BOOT button; active-low
SELECT switch	`SELECT_GPIO`	`10`	in (PU)	IN = GPIO→GND (active-low)
Legend OFF (white)	`LEGEND_OFF_GPIO`	`21`	out	drives top legend half (via driver)
Legend FAULT (amber)	`LEGEND_FAULT_GPIO`	`14`	out	drives bottom legend half (via driver)
Status LED	`STATUS_LED_GPIO`	`48`	out	on-board RGB on most S3 devkits
Mic bit clock	`MIC_BCLK_GPIO`	`4`	out	I2S0 BCLK → mic SCK
Mic word select	`MIC_LRCLK_GPIO`	`5`	out	I2S0 WS → mic WS
Mic data in	`MIC_DIN_GPIO`	`6`	in	mic SD → ESP DIN
SD clock	`SD_CLK_GPIO`	`7`	out	SDMMC CLK
SD command	`SD_CMD_GPIO`	`9`	i/o	SDMMC CMD (needs pull-up)

Function	Macro	GPIO	Dir	Notes
SD data 0	`SD_D0_GPIO`	8	i/o	SDMMC DAT0 (needs pull-up)
Speaker bit clock	`SPK_BCLK_GPIO`	15	out	I2S1 BCLK → amp BCLK
Speaker word select	`SPK_LRCLK_GPIO`	16	out	I2S1 WS → amp LRC
Speaker data out	`SPK_DOUT_GPIO`	17	out	I2S1 DOUT → amp DIN

Polarity macros (flip to match your hardware): `PTT_ACTIVE_LOW=1`, `SELECT_ACTIVE_LOW=1`, `LEGEND_OFF_ACTIVE_HIGH=1`, `LEGEND_FAULT_ACTIVE_HIGH=1`, `STATUS_LED_ACTIVE_HIGH=1`, `LAMP_TEST_MS=2000`. Set any unused output to `-1` to disable it cleanly.

Reserved / avoid pins (ESP32-S3-WROOM-1 N16R8)

Pins	Reason
`GPIO33-37`	Octal PSRAM/flash internal bus — do not use on R8 modules
`GPIO19, GPIO20`	USB D-/D+ (native USB / USB-Serial-JTAG)
`GPIO0, 45, 46`	Strapping pins — usable but must be in the correct state at boot
`GPIO26-32`	SPI flash on some modules — verify your board

GPIO0 is used here only as the BOOT/PTT button (its natural strapping use), so it is safe. None of the chosen pins conflict with the reserved set.

3 Per-Device Wiring

3.1 INMP441 MEMS Microphone → ESP32-S3 (I2S_NUM_0)

Supply 1.8–3.3 V (never 5 V), ~2.2–2.5 mA at 3.3 V. INMP441 datasheet.

INMP441 pin	Connects to	Net
VDD	ESP 3V3	3.3 V
GND	ESP GND	GND
SCK	ESP GPIO4 (BCLK)	mic I2S
WS	ESP GPIO5 (WS)	mic I2S
SD	ESP GPIO6 (DIN)	mic I2S
L/R	GND	selects left channel

- Decouple VDD→GND with 0.1 µF close to the module.
- Add a 100 kΩ pulldown on the SD line (datasheet-recommended to discharge the bus when tri-stated). Most breakouts include it.
- Do not clock WS/SCK with VDD unpowered (stresses ESD diodes).

3.2 MAX98357A Class-D Amplifier → ESP32-S3 (I2S_NUM_1)

Supply 2.5–5.5 V; 2.4 mA quiescent; peak speaker current up to ~650 mA at 5 V/4 Ω. No MCLK required. Analog Devices datasheet, Adafruit guide.

MAX98357A pin	Connects to	Net
VIN	5 V (USB/VBUS) for full output (3.3 V also works, less power)	5 V
GND	ESP GND	GND
BCLK	ESP GPIO15	spk I2S
LRC	ESP GPIO16	spk I2S
DIN	ESP GPIO17	spk I2S
GAIN	NC = 9 dB (default)	—
SD (mode)	float = mono (L+R)/2	—
OUT+/OUT-	speaker (4–8 Ω) — bridge-tied, no ground ref	—

- GAIN options: 100 kΩ→GND = 15 dB; →GND = 12 dB; NC = 9 dB; →VIN = 6 dB; 100 kΩ→VIN = 3 dB.
- SD/mode pin: <0.16 V = shutdown; 0.16–0.77 V = (L+R)/2 mono; 0.77–1.4 V = right; >1.4 V = left. Breakouts usually float it to mono.
- The OUT pins are bridge-tied — never connect either to GND.

3.3 microSD Card → ESP32-S3 (SDMMC, 1-bit)

3.3 V card. Sleep ~100–200 µA; init/read peaks 50–200 mA. SD current notes.

SD signal	ESP32-S3	Net	Pull-up
CLK	GPIO7	microSD	—
CMD	GPIO9	microSD	10 kΩ → 3V3

SD signal	ESP32-S3	Net	Pull-up
DAT0	GPIO8	microSD	10 kΩ → 3V3
VDD	3V3	3.3 V	—
VSS	GND	GND	—

- 1-bit mode uses only DAT0 (DAT1–3 unused). For 4-bit, add DAT1/2/3 with pull-ups.
- Keep CLK trace short; SDMMC runs at MHz clocks. Format FAT32.

3.4 Discrete Inputs

Input	ESP32-S3	Active	Idle
SELECT switch (IN/OUT)	GPIO10	LOW = selected IN	internal PU → HIGH = OUT
Push-to-talk	GPIO0	LOW = pressed	internal PU → HIGH

Wire each as a simple SPST contact to GND. No external resistor needed (internal pull-ups enabled in firmware). Debounce is handled in software.

4 Annunciator Switch & Lamp Driver

Applied Avionics split-legend switch with two independently driven legend halves, following dark-cockpit philosophy per FAA AC 25-11B and the AEA design guidance.

State	TOP — white VOICE CHKLST OFF	BOTTOM — amber VOICE CHKLST FAULT
Selected IN, healthy	dark	dark ← true dark cockpit
Selected IN, fault	dark	amber ON
Selected OUT	white ON	dark (fault inhibited)

- Power-up lamp test: both halves on for `LAMP_TEST_MS` (~2 s) so a dead LED cannot be mistaken for a healthy dark state, then dark.
- OUT inhibits fault: a deliberately deselected system needs no crew action, so AC 25-11 keeps the amber caution off; only the white OFF status shows. Recognition and fault monitoring are suspended while OUT.

4.1 VIVISUN Voltage Options

Applied Avionics VIVISUN / Korry lighted pushbuttons are offered in 28 VDC, 5 VDC, 28 VAC, 5 VAC, 115 V lamp variants. Applied Avionics. An ESP32 GPIO cannot drive a 28 V (or even 5 V at lamp current) legend directly — use a low-side driver per half (below). For a quick bench mock-up, substitute two ordinary 3.3 V LEDs (white + amber) with series resistors driven straight from GPIO21/GPIO14, observing the ~20 mA GPIO limit.

4.2 Lamp-Driver Circuit (one per legend half)

ESP32-S3 GPIO sources ~20 mA default (40 mA max, configurable), with a 1.5 A total chip limit. ESP32 GPIO current. Legend lamps need their own supply and a switch device:

```
              +V_lamp (5 V or 28 V, separate rail)
                    |
                [ legend lamp ]        <- VIVISUN legend (white or amber)
                    |
                    +------------------ Drain
     GPIO21 --[1k]--|G   N-ch MOSFET (logic-level, e.g. 2N7002 / AO3400)
     (or G14)       |    or NPN (e.g. 2N2222 with base resistor)
                [10k]                  <- gate/base pulldown -> GND (defined OFF)
                    |
                   GND  (Source) ------ common ground with ESP32
```

- One driver for GPIO21 (OFF/white half), one for GPIO14 (FAULT/amber half).
- 10 kΩ gate/base pulldown guarantees the lamp is OFF during boot/reset before the GPIO is configured (dark-cockpit integrity).
- Choose a MOSFET/transistor and supply rated for the lamp. For a 28 V incandescent legend, ensure FET Vds ≥ 40 V and current ≥ lamp inrush.
- Tie the lamp supply ground to the ESP32 ground (single common ground).

5 Power Budget & Supply

Rail	Loads	Typical	Peak
3.3 V	ESP32-S3 + Wi-Fi off + mic + microSD	~80–150 mA	~250 mA (SD init / SR burst)
5 V	MAX98357A speaker output	a few mA idle	~650 mA (5 V/4 Ω, loud)
Lamp rail	up to 2 legend halves	0 (dark)	per lamp spec (e.g. 28 V incand.)

- Power the board from USB 5 V capable of ≥ 1 A (the on-board 3.3 V LDO feeds the S3, mic, and SD). Reserve headroom for the amp's peak.
- Keep the legend-lamp supply separate from the logic 5 V if using 28 V lamps; only the grounds are common.
- Add bulk decoupling: ≥ 100 µF near the amp VIN plus 0.1 µF per device.

6 Bill of Materials (DIY build)

Qty	Part	Spec / example	Approx.
1	ESP32-S3 DevKit	DevKitC-1 N16R8 (PSRAM!)	$12–18
1	I2S MEMS mic	INMP441 or ICS-43434 breakout	$4–6
1	I2S amp	MAX98357A breakout	$4–7
1	Speaker	4–8 Ω, ≥ 2 W	$2–5
1	microSD card + breakout	FAT32; Korvo-2 has on-board slot	$5–8
1	Annunciator switch	Applied Avionics VIVISUN/Korry split-legend (or 2 LEDs for bench)	varies
2	Lamp driver	logic-level N-MOSFET (2N7002/AO3400) or NPN (2N2222)	<$1
4	Resistors	1 kΩ ×2 (gate), 10 kΩ ×2 (pulldown)	<$1
2–3	Pull-ups	10 kΩ on SD CMD/DAT0 (if breakout lacks them)	<$1
—	Caps	0.1 µF per device, 100 µF bulk near amp	<$1
1	PTT button	momentary SPST (or use BOOT)	<$1

Integrated alternative: ESP32-S3-Korvo-2 (~$45–55) replaces the mic/codec/amp/SD items above; use its BSP pin map and the ES8311 codec init.

7 ESP32-S3-Korvo-2 Differences (integrated build)

- Audio codec is ES8311 (I2C control + I2S data) with an NS4150 speaker amp — not the MAX98357A path. Initialize the codec over I2C (see `esp_codec_dev`/BSP).
- Dual mics feed the AFE — markedly better recognition in noise than one INMP441.
- microSD slot is wired on the board; map `SD_*` pins to the Korvo-2 schematic.

- Expose the annunciator on free header GPIOs; keep the same firmware logic.

8 Firmware Build & Behavior

Topic	Value
Framework	ESP-IDF ≥ 5.2
Components	`esp-sr, esp_spiffs, driver, json (cJSON), fatfs, sdmmc, esp_driver_sdmmc`
Speech models	WakeNet `WN9_HIESP`; MultiNet English `mn6_en`/`mn7_en` (S3)
Partitions	factory app 3 MB + model 5 MB + storage (UI clips) 2 MB → needs 16 MB flash
MultiNet phrases	lowercase letters + single spaces only; spell numbers ("v one"); ~200 cmd cap
Fault behavior	any SD/parse/validation/audio fault → `ST_FAULT`, amber legend, no checklist shown

> The partition table assumes a 16 MB flash part (N16R8). An 8 MB module will not fit the 3 MB app + 5 MB model + 2 MB storage layout.

8.1 Build & Flash

```
cd firmware
# 1) Render the spoken checklist clips into ./audio (pick your TTS engine):
python3 scripts/make_audio.py --engine say        # macOS
#   or  --engine espeak    or  --engine piper --voice
# 2) Target the S3 and build/flash (models + audio embedded automatically):
idf.py set-target esp32s3
idf.py menuconfig          # optional - defaults select Hi ESP + mn6_en
idf.py build flash monitor
```

8.2 State Machine & Fault Codes

The firmware mirrors the web demo with four states. `HOME` recognises a checklist trigger and loads it; `RUN` recognises each item's advance word and steps forward. To keep recognition fast, only the phrases relevant to the current step are registered (`esp_mn_commands_add` + `esp_mn_commands_update`).

State	Meaning
`ST_OFF`	SELECT switched OUT — white OFF legend lit, voice ignored, fault monitoring suspended
`ST_FAULT`	SD/data/audio problem — amber FAULT legend lit, cause annunciated, retries loading
`ST_HOME`	Healthy & idle — listening for a checklist trigger word
`ST_RUN`	Reading a checklist — listening for the active item's advance word

Per the FAA-aligned safety rule, `checklist_store_load()` returns a specific fault code on any problem and leaves the table empty; the device never presents a partial or stale list. The UI/fault clips live in internal SPIFFS (not the card) so it can still speak a fault with no card inserted.

Fault code	Trigger
`STORE_OK`	SD mounted, JSON parsed + validated, audio present (healthy)
`FAULT_NO_CARD`	SD not detected / mount failed

Fault code	Trigger
`FAULT_NO_AIRCRAFT`	no aircraft folder (or config.txt target missing)
`FAULT_NO_JSON`	checklists.json missing or unreadable
`FAULT_PARSE`	JSON malformed
`FAULT_VALIDATION`	schema / voice-rule violation, empty data, etc.
`FAULT_AUDIO_MISSING`	a referenced read-aloud clip is absent
`FAULT_NO_MEMORY`	allocation failed while loading

9 Processor Selection

9.1 What this application actually needs

This is not a general dictation device. The workload is narrow and well bounded:

Requirement	Implication
Small, fixed vocabulary	~13 checklist triggers + a handful of universal advance words = well under the ~200-command MultiNet cap. No large-vocabulary STT needed.
Offline, no connectivity	A safety/training device should not depend on WiFi, BT, or cloud. Connectivity is a liability, not a feature.
Deterministic + safe	Must revert to a safe (unopened) state on any fault; single-chip, well-understood toolchain reduces failure surface.
Dark-cockpit annunciation	A few GPIO + I2S in/out + SD. No high-end peripherals required.
Low cost, mature tooling	Demo/training hardware; reproducible with off-the-shelf parts.

The net: command-recognition on a fixed grammar, not open-ended transcription. That keeps an MCU-class part firmly in scope and makes a Linux SBC overkill.

9.2 Recommendation

> Keep the ESP32-S3 as the baseline; consider the ESP32-P4 only if you want more headroom. ESP-SR (v2.1+) explicitly supports S3 and P4 for English MultiNet, running wake word + AEC/NS + intent on a single chip. The classic ESP32 is no longer supported by current speech algorithms and should be avoided.

- ESP32-S3 (recommended baseline): Xtensa LX7 dual-core @240 MHz with AI vector instructions, 512 KB SRAM, WiFi+BLE, ~$8–15. Rated the best AI/voice part in the ESP line, most mature tooling, runs `mn6_en`/`mn7_en`. The firmware, wiring diagram, and BOM are already built around it. Espressif ESP-SR, espboards.dev.
- ESP32-P4 (upgrade path): dual-core RISC-V up to 400 MHz + AI instructions + a 40 MHz low-power core, 768 KB SRAM, ~2.5× the compute of the S3, runs `mn7_en`. No WiFi/BT — for a safety device, arguably a plus (removes an attack/distraction surface). Trade-off: needs a companion radio for any connectivity, and slightly less mature tooling. Good if you later add a display or more audio processing. Espressif ESP32-P4, Elecrow P4 vs S3.

For this fixed-vocabulary workload the S3 has ample margin, so the P4 is a “want more headroom / future display” upgrade rather than a necessity.

9.3 Alternatives Considered

Option	What it is	Verdict for this app
Syntiant NDP120 (Arduino Nicla Voice)	Always-on Neural Decision Processor, ultra-low-power, embedded Cortex-M0	Excellent for battery always-on wake-word; overkill here since we have panel power and need full command grammar + audio playback + SD.

Option	What it is	Verdict for this app
Picovoice Porcupine + Rhino	Wake-word + speech-to-intent, offline on Arm Cortex-M4	Clean fit for fixed grammar, but requires a per-deployment AccessKey (license dependency) — undesirable for a self-contained demo.
Fluent.ai	End-to-end speech-to-intent on Cortex-M4 @100 MHz, multilingual	Good for productized multilingual intent; commercial licensing, less open tooling than ESP-SR.
NXP i.MX RT600	Cortex-M33 @300 MHz + Cadence HiFi4 DSP @600 MHz, 4.5 MB SRAM	Strong audio DSP, but more complex board + toolchain than needed for a fixed 13-checklist grammar.
Raspberry Pi (whisper.cpp / Vosk)	Full offline STT on Linux	Real large-vocabulary transcription, but Linux boot, higher power/cost, non-deterministic boot — overkill and less robust for a fixed-grammar safety device.

Bottom line: the ESP32-S3 remains the right baseline; the ESP32-P4 is the only “strictly better” silicon in the same family and is worth it only if you want extra compute or a display. The dedicated voice chips (Syntiant, Picovoice, Fluent.ai) and the Pi solve problems this app doesn't have.

10 microSD Configuration & JSON Schema

Checklist data is loaded from the card at boot. There is no compiled-in fallback — data comes only from the card. If the card has exactly one aircraft folder it loads automatically; `config.txt` is only needed when multiple folders exist.

```
/sdcard/
  config.txt                 (optional) one line:  AIRCRAFT=CJ2
  CJ2/
    checklists.json          the checklist data (schema below)
    audio/
      engine_fire_1.wav      one 16-bit/16 kHz/mono WAV per item "clip"
      engine_fire_2.wav
      ready.wav  complete.wav ...
```

checklists.json schema

```
{
  "aircraft": "CJ2",
  "universal_advance": ["check", "checked", "complete", "next"],
  "checklists": [
    {
      "id": "engine_fire",
      "title": "Engine Fire",
      "triggers": ["engine fire"],
      "items": [
        { "clip": "engine_fire_1",
          "text": "Throttle ... idle",
          "advance": ["idle"] }
      ]
    }
  ]
}
```

> Voice-phrase rule (all triggers, advance, universal_advance): lowercase letters and single spaces only — no digits or punctuation (MultiNet limitation). Spell numbers as words (“v one”). Limits: ≤ MAX_ITEMS (12) items, ≤ MAX_TRIGGERS (4) triggers, ≤ MAX_ADVANCE (4) advance words per item. The loader validates the whole file and verifies every audio clip exists before going live; any violation = FAULT.

11 CJ2 Checklist Library

Aircraft CJ2 · 13 checklists. Universal advance words (accepted on any item): `check`, `checked`, `complete`, `next`.

Numbers spelled as words for MultiNet (e.g. "v one" = V-one). Each item lists its read-aloud text and the spoken word(s) that advance it.

Engine Fire

Triggers: `engine fire` · `fire warning`

#	Read-aloud item	Advance word(s)
1	Throttle, affected engine, idle.	`idle`
2	Engine fire button, lift cover and push.	`push / pushed`
3	Either illuminated bottle armed button, push.	`push / pushed / armed`

Takeoff Rejected — Below V1

Triggers: `takeoff rejected` · `abort takeoff` · `speed below v one`

#	Read-aloud item	Advance word(s)
1	Brakes, as required.	`as required / required`
2	Throttles, idle.	`idle`
3	Speed brakes, extend.	`extend / extended`

Takeoff Continued — Above V1

Triggers: `takeoff continued` · `speed above v one`

#	Read-aloud item	Advance word(s)
1	Maintain directional control.	`control / maintained`
2	Accelerate to V R.	`accelerating / rotate speed`
3	Rotate at V R, climb at V two.	`rotate / climbing`
4	Landing gear up, after positive rate of climb.	`up / gear up`
5	At fifteen hundred feet, retract flaps, then accelerate.	`flaps up / accelerating`

Engine Failure During Final Approach

Triggers: `engine failure final approach` · `engine failure approach`

#	Read-aloud item	Advance word(s)
1	Thrust, operating engine, increase as required.	`increased / as required`
2	Airspeed, V approach.	`v approach / set`
3	Flaps, takeoff and approach.	`set / flaps set`

Emergency Restart — Two Engines

Triggers: `emergency restart` · `engine restart` · `restart engines`

#	Read-aloud item	Advance word(s)
1	Ignition switches, both on.	`both on / on`
2	Left and right fuel boost switches, both on.	`both on / on`
3	Throttles, idle.	`idle`
4	If altitude allows, increase airspeed to two hundred knots.	`increased / set`

Electrical Fire or Smoke

Triggers: `electrical fire` · `electrical smoke` · `smoke removal` · `cabin smoke`

#	Read-aloud item	Advance word(s)
1	Oxygen masks, don and emergency.	`on / donned / emergency`
2	Microphone select switches, mic oxygen mask.	`set / selected`

Cabin Altitude

Triggers: `cabin altitude` · `cabin alt` · `depressurization`

#	Read-aloud item	Advance word(s)
1	Oxygen masks, don and one hundred percent oxygen.	`on / donned`
2	Microphone select switches, mic oxygen mask.	`set / selected`
3	Emergency descent, as required.	`as required / descending`
4	Passenger oxygen, make sure passengers are receiving oxygen.	`confirmed / receiving`

Emergency Descent

Triggers: `emergency descent` · `rapid descent`

#	Read-aloud item	Advance word(s)
1	Autopilot trim disconnect button, press and release.	`pressed / released`
2	Throttles, idle.	`idle`
3	Speed brakes, extend.	`extend / extended`
4	Airplane pitch attitude, approximately twenty degrees nose down.	`nose down / set`

Battery Overtemperature

Triggers: `battery overtemp` · `battery overtemperature` · `batt o temp`

#	Read-aloud item	Advance word(s)
1	Volt amp, note.	`noted / note`
2	Battery switch, emergency.	`emergency / set`
3	Volt amp, note decrease.	`noted / decreasing`

Autopilot Malfunction

Triggers: `autopilot malfunction · autopilot failure`

#	Read-aloud item	Advance word(s)
1	Autopilot trim disconnect button, press and release.	pressed / released

Electric Elevator Trim Runaway

Triggers: `trim runaway · elevator trim runaway · elevator trim`

#	Read-aloud item	Advance word(s)
1	Autopilot trim disconnect button, press and release.	pressed / released
2	Throttles, as required.	as required / set
3	Speed brakes, as required.	as required / set
4	Manual elevator trim, as required.	as required / set
5	Pitch trim circuit breaker, left panel, pull.	pull / pulled

Emergency Evacuation

Triggers: `emergency evacuation · evacuate · evacuation`

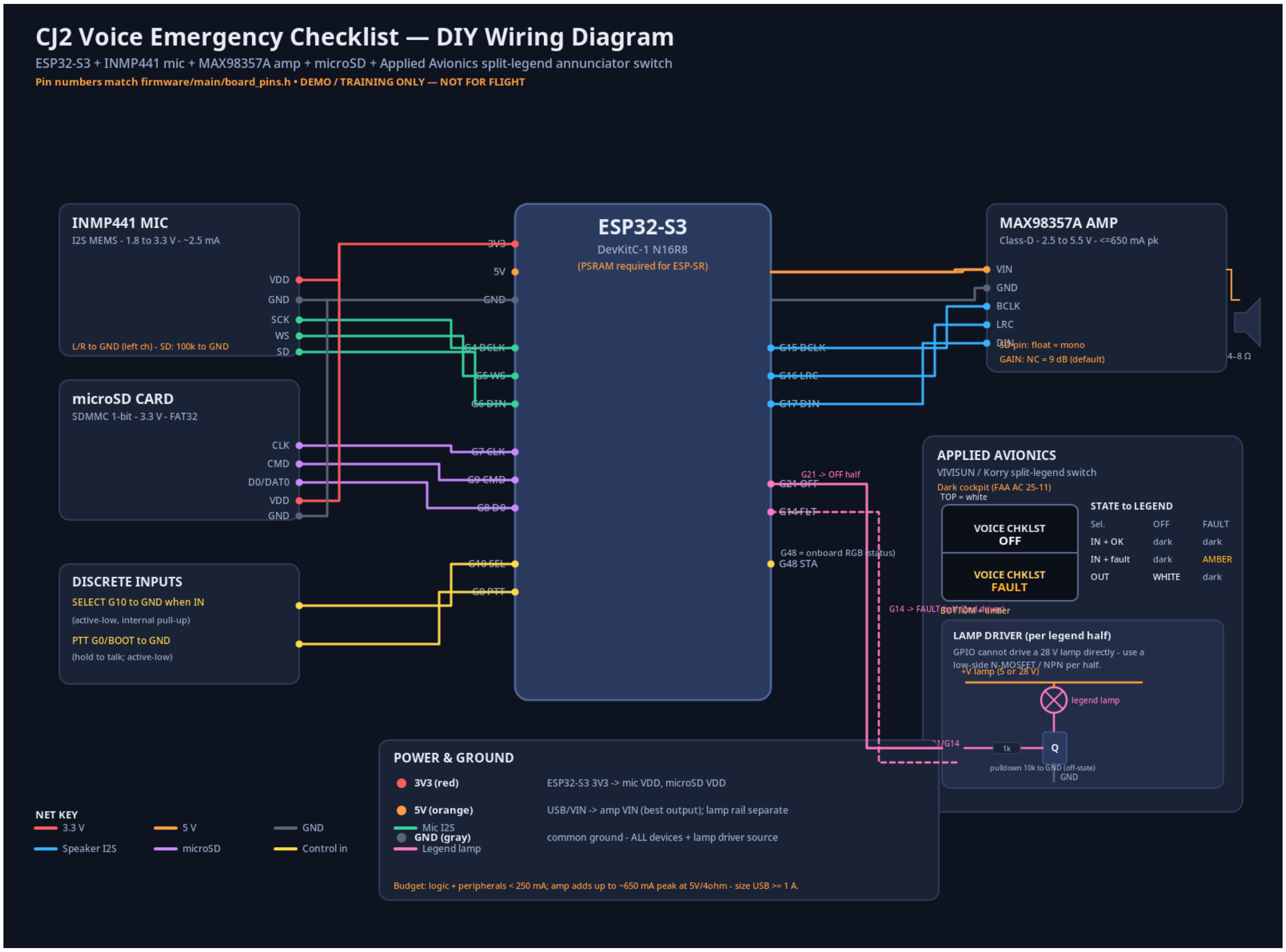
#	Read-aloud item	Advance word(s)
1	Parking brake, set.	set
2	Throttles, both off.	both off / off
3	Left and right engine fire buttons, both press.	both press / pressed
4	Illuminated bottle armed buttons, both press, if fire suspected.	both press / pressed
5	Battery switch, off.	off
6	Emergency locator transmitter, make sure the system is activated.	activated / confirmed
7	Airplane and immediate area, check for the best escape route.	checked / confirmed
8	If exiting through the cabin door, open it. If through the emergency exit, remove and throw the door clear.	cabin door / emergency exit
9	Move away from the airplane. Procedure complete.	clear / complete

Before Start

Triggers: `before start · before engine start`

#	Read-aloud item	Advance word(s)
1	Parking brake, set.	set
2	Seat belts, fasten.	fastened / set
3	Battery switch, on.	on
4	Fuel quantity, check.	checked / check
5	Flight controls, free and correct.	free and correct / checked

12 Wiring Diagram



Color-coded schematic: ESP32-S3 + INMP441 mic + MAX98357A amp + microSD + Applied Avionics split-legend annunciator with the lamp-driver sub-circuit and dark-cockpit state table.

13 Source References

#	Reference	Source
1	INMP441 microphone datasheet	Farnell / InvenSense
2	MAX98357A amplifier datasheet	Analog Devices
3	MAX98357A breakout guide	Adafruit
4	ESP32-S3 GPIO drive current	Espressif forum
5	ESP32-S3 GPIO reference	Espressif docs
6	ESP32-S3 reserved/strapping pins (R8 octal)	Arduino forum summary
7	microSD operating current	Arduino forum
8	VIVISUN lamp voltage options	Applied Avionics
9	Annunciation color / dark-cockpit guidance	FAA AC 25-11B
10	Avionics display design article	AEA Avionics News
11	ESP-SR speech recognition framework	Espressif docs
12	ESP32 SoC options for AI/voice	espboards.dev
13	ESP32-P4 product page	Espressif
14	ESP32-P4 vs ESP32-S3 performance	Elecrow
15	Syntiant NDP120 Neural Decision Processor	Syntiant
16	Keyword spotting on microcontrollers	Picovoice